

scripts/03_identify_model.py

176 lines

```
1 """
2 03_identify_model.py
3 =====
4 Dynamic model identification from the IDENTIFICATION step-test data only
5 (`data/step_test_identification.csv`), then validation against the held-out
6 VALIDATION step-test data (`data/step_test_validation.csv`).
7
8 For each output y in {Q, WHP, FLP, BHP}:
9 * steady-state knots y_ss(u) are read directly from the end-of-step values
10 of the identification test (piecewise-linear steady-state curve)
11 * a single first-order time constant tau_y is fitted by nonlinear least
12 squares against the *shape* of every step's transient (normalised to
13 0-1), pooled across all steps for robustness
14 * dead time theta is fitted too (expected ~0 for this plant) for honesty
15
16 Outputs
17 -----
18 data/fitted_model.json (the WellModel, used by the controller)
19 data/model_gain_table.csv (local gain vs operating point, for report)
20 figures/model_validation.png (predicted vs actual on held-out data)
21 data/model_validation_metrics.json (RMSE / NRMSE per output)
22 """
23
24 from __future__ import annotations
25
26 import json
27 import sys
28 from pathlib import Path
29
30 sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))
31
32 import numpy as np
33 import pandas as pd
34 from scipy.optimize import least_squares
35 import matplotlib.pyplot as plt
36
37 from config import (DATA_DIR, FIG_DIR, STEP_HOLD_HOURS, STEP_SEQUENCE_ID,
38 STEP_SEQUENCE_VAL, TS_HOURS)
39 from model import WellModel, OutputModel, OUTPUTS
40 from plotting import COLOR_Q, COLOR_WHP, COLOR_FLP, COLOR_BHP, style_axis, savefig
41
42 COLORS = {"Q": COLOR_Q, "WHP": COLOR_WHP, "FLP": COLOR_FLP, "BHP": COLOR_BHP}
43
44
45 def extract_steady_state_knots(df: pd.DataFrame, sequence, hold_hours, y_col):
46     """Steady-state value of `y_col` at the END of every held step."""
47     u_knots, y_knots = [], []
48     for i, u in enumerate(sequence):
49         # last sample of this hold window (steady state, since hold >> tau)
50         end_row = df[df["step_index"] == i].iloc[-1]
51         u_knots.append(u)
52         y_knots.append(end_row[y_col])
53     # de-duplicate / sort by u for a well-posed interpolant
54     order = np.argsort(u_knots)
55     u_sorted = np.array(u_knots)[order]
56     y_sorted = np.array(y_knots)[order]
57     u_uniq, idx = np.unique(u_sorted, return_index=True)
58     return u_uniq.tolist(), y_sorted[idx].tolist()
59
60
61 def fit_time_constant(df: pd.DataFrame, sequence, hold_hours, y_col, u_knots, y_knots):
62     """
63     Pool every step's transient (normalised to its own start/end) and fit one
64     (tau, theta) pair by nonlinear least squares against the first-order
65     step-response shape y_norm(t) = 1 - exp(-(t-theta)/tau).
66     """
67     t_rel_all, y_norm_all, tau_per_step = [], [], []
```

```

68
69 for i in range(1, len(sequence)):
70     seg = df[df["step_index"] == i].reset_index(drop=True)
71     y0 = df[df["step_index"] == i - 1].iloc[-1][y_col]
72     y1 = seg.iloc[-1][y_col]
73     if abs(y1 - y0) < 1e-6:
74         continue
75     t_rel = np.arange(len(seg)) * TS_HOURS
76     y_norm = (seg[y_col].values - y0) / (y1 - y0)
77
78 # quick per-step tau estimate (time to reach 63.2%) for diagnostics
79 idx_63 = np.searchsorted(y_norm, 0.632)
80 if 0 < idx_63 < len(t_rel):
81     tau_per_step.append(float(t_rel[idx_63]))
82
83 t_rel_all.append(t_rel)
84 y_norm_all.append(y_norm)
85
86 t_all = np.concatenate(t_rel_all)
87 y_all = np.concatenate(y_norm_all)
88
89 def resid(p):
90     tau, theta = p
91     tau = max(tau, 0.05)
92     theta = max(theta, 0.0)
93     pred = 1.0 - np.exp(-np.maximum(t_all - theta, 0.0) / tau)
94     return pred - y_all
95
96 res = least_squares(resid, x0=[1.0, 0.0], bounds=([0.05, 0.0], [10.0, 2.0]))
97 tau_fit, theta_fit = res.x
98 rmse = float(np.sqrt(np.mean(res.fun ** 2)))
99 return float(tau_fit), float(theta_fit), rmse, tau_per_step
100
101
102 def identify(df_id: pd.DataFrame) -> WellModel:
103     outputs = {}
104     for y in OUTPUTS:
105         u_knots, y_knots = extract_steady_state_knots(
106             df_id, STEP_SEQUENCE_ID, STEP_HOLD_HOURS, y
107         )
108         tau, theta, rmse, tau_per_step = fit_time_constant(
109             df_id, STEP_SEQUENCE_ID, STEP_HOLD_HOURS, y, u_knots, y_knots
110         )
111         outputs[y] = OutputModel(
112             name=y, tau=tau, theta=theta, u_knots=u_knots, y_knots=y_knots,
113             tau_per_step=tau_per_step, rmse_fit=rmse,
114         )
115     print(
116         f" {y:4s}: tau={tau:.2f} h theta={theta:.2f} h "
117         f"fit RMSE(normalised)={rmse:.3f} "
118         f"per-step tau spread={np.round(tau_per_step, 2)}"
119     )
120     return WellModel(ts=TS_HOURS, outputs=outputs)
121
122
123 def validate(model: WellModel, df_val: pd.DataFrame, sequence, hold_hours):
124     """Open-loop prediction from the model over the held-out validation test,
125     starting from the true initial condition, applying the same choke sequence,
126     and comparing to the measured (noise-free) plant trajectory."""
127     y0 = {y: float(df_val.iloc[0][y]) for y in OUTPUTS}
128     u_seq = df_val["choke_pct"].values[1:] # skip the initial reset row
129     pred = model.predict(y0, u_seq)
130
131     actual = {y: df_val[y].values[1:] for y in OUTPUTS}
132     t = df_val["time_h"].values[1:]
133
134     metrics = {}
135     fig, axes = plt.subplots(4, 1, figsize=(11, 10), sharex=True)
136     for ax, y in zip(axes, OUTPUTS):
137         rmse = float(np.sqrt(np.mean((pred[y] - actual[y]) ** 2)))

```

```

138 span = max(actual[y].max() - actual[y].min(), 1e-6)
139 nrmse = 100.0 * rmse / span
140 metrics[y] = {"rmse": rmse, "nrmse_pct": nrmse}
141
142 ax.plot(t, actual[y], color=COLORS[y], lw=2.0, label="Actual (plant)")
143 ax.plot(t, pred[y], color="black", lw=1.4, ls="--", label="Predicted (model)")
144 style_axis(ax, title=f"{y}: RMSE={rmse:.2f}, NRMSE={nrmse:.1f}%", ylabel=y)
145 ax.legend(loc="best", fontsize=8)
146 axes[-1].set_xlabel("Time [h]")
147 fig.suptitle("Model Validation on Held-Out Step Test", fontweight="bold")
148 fig.tight_layout()
149 savefig(fig, FIG_DIR / "model_validation.png")
150 return metrics
151
152
153 def main():
154     df_id = pd.read_csv(DATA_DIR / "step_test_identification.csv")
155     df_val = pd.read_csv(DATA_DIR / "step_test_validation.csv")
156
157     print("Identifying model from identification step test...")
158     model = identify(df_id)
159     model.save(DATA_DIR / "fitted_model.json")
160     print(f" wrote {DATA_DIR / 'fitted_model.json'}")
161
162     print("\nValidating against held-out step test...")
163     metrics = validate(model, df_val, STEP_SEQUENCE_VAL, STEP_HOLD_HOURS)
164     for y, m in metrics.items():
165         print(f" {y:4s}: RMSE={m['rmse']:.2f} NRMSE={m['nrmse_pct']:.1f}%")
166     (DATA_DIR / "model_validation_metrics.json").write_text(json.dumps(metrics, indent=2))
167
168     gain_table = model.gain_table([10, 20, 30, 40, 50, 60, 70, 80, 90])
169     pd.DataFrame(gain_table).to_csv(DATA_DIR / "model_gain_table.csv", index=False)
170     print(f"\n wrote {DATA_DIR / 'model_gain_table.csv'}")
171     print(f" wrote {FIG_DIR / 'model_validation.png'}")
172
173
174 if __name__ == "__main__":
175     main()
176

```